

Reviewer Pack — Minimal Rank of Noncrossing Partitions Bounds Resolution Pro...

Entry 88dea7710909 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 03:23:53 UTC

Minimal Rank of Noncrossing Partitions Bounds Resolution Proof Length

Entry ID: 88dea7710909 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 03:23:53 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Noncrossing Partitions) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For a given Tseitin formula F on n variables, the minimal rank of its associated noncrossing partition P is such that the Resolution proof length for F is at least $2^{(\Omega(\text{rank}(P)))}$ ’, ‘sentence_2’: ‘This holds with high probability for random Tseitin formulas.’, ‘sentence_3’: ‘The minimal rank of a noncrossing partition can be computed in polynomial time.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Noncrossing partitions encode combinatorial properties that are believed to be hard for resolution algorithms, suggesting a potential link to the complexity of Resolution proofs.’, ‘sentence_2’: ‘The structure of noncrossing partitions may reveal hidden complexities in Tseitin formulas that are not apparent through standard circuit complexity measures.’, ‘sentence_3’: ‘This conjecture could provide new insights into the nature of proof systems and their limitations.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 166f34cd62b8bf04

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Tseitin formulas with $n \leq 40$ variables, computed minimal rank of noncrossing partition P satisfies $2^{(\Omega(\text{rank}(P)))} \leq \text{Resolution proof length}$ AND at least 80% of seeds produce a resolution proof length within a factor of $2^{(0.5 * \text{rank}(P))}$ of the expected value.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "noncrossing partitions" AND "Resolution proof complexity" - "minimal rank" AND "Tseitin formula" AND "proof length" - "computability" AND "noncrossing partitions" AND "resolution complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        literals = [f"x{i}" for i in range(1, n+1)]
        clauses = []
        for literal in literals:
            clauses.append([literal])
        for i in range(1, n):
            for j in range(i+1, n+1):
                clauses.append([f"~{literals[i-1]}", f"{literals[j-1]}"])
                clauses.append([f"~{literals[j-1]}", f"{literals[i-1]}"])
        return literals, clauses

    def compute_noncrossing_partition(literals, clauses):
        n = len(literals)
        partition = [set() for _ in range(n+1)]
        for clause in clauses:
            if len(clause) == 2 and clause[0].startswith("~") and clause[1] != clause[0][1:]:
                i = int(clause[0][1:]) - 1
                j = int(clause[1]) - 1
                if i < j:
                    partition[i+1].add(j)
        return partition

    def resolution_proof_length(literals, clauses):
```

```

n = len(literals)
stack = []
for clause in clauses:
    stack.append(clause)

while stack:
    clause = stack.pop()
    if not any(literal in clause for literal in literals):
        continue
    new_clause = set()
    for other_clause in stack:
        common_lits = [lit for lit in clause if lit.startswith("~") and lit[1:] in other_clause]
        if len(common_lits) == 2:
            new_lit = f"{common_lits[0][1:]} {common_lits[1][1:]}"
            if new_lit not in literals:
                return float('inf')
            new_clause.add(new_lit)
    stack.append(list(new_clause))

return len(stack)

n = random.choice([5, 10, 15, 20, 30, 40])
literals, clauses = generate_tseitin_formula(n)
partition = compute_noncrossing_partition(literals, clauses)

rank_P = max(len(x) for x in partition if x)
proof_length = resolution_proof_length(literals, clauses)

metric_value = proof_length
conjecture_holds = proof_length >= 2 ** (math.log2(rank_P))
counterexample = "" if conjecture_holds else f"rank(P)={rank_P}, proof_length={proof_length}"

return {
    "metric_name": "Resolution Proof Length",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    total_metric = sum(r["metric_value"] for r in results if r["conjecture_holds"])
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric/len(results)} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:

```

```

print(f"RESULT: SUPPORTED mean={total_metric/len(results)} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7c6e988.py", line 90, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7c6e988.py", line 67, in run_trial
    partition = compute_noncrossing_partition(literals, clauses)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a7c6e988.py", line 37, in compute_noncrossing_partition
    i = int(clause[0][1:]) - 1
        ^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'x1'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11060
2	propose	ollama_remote	glm4:latest	0	0	12365
3	preregistration	ollama_remote	glm4:latest	0	0	6196
4	novelty	ollama_remote	glm4:latest	0	0	4859
5	novelty	ollama_remote	glm4:latest	0	0	9995
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18266
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10730
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9331
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11813

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	16635

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111251 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/88dea7710909.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/88dea7710909.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/88dea7710909.tar.gz (if generated)